



Q1 String Character Frequency Counter

10 marks

Write a Python program that accepts a string from the user and counts the frequency of each unique character (case-insensitive). Display the results in the format: character → count. Sort the output alphabetically by character.

Constraints:

- Ignore spaces in the count.
- Treat uppercase and lowercase as the same character.
- Sort output alphabetically.

Skills Assessed:

- String iteration, lower(), conditionals
- Dictionary or manual counting
- Sorting using loops or sorted()

Input: Hello World

output:

d → 1

e → 1

h → 1

l → 3

o → 2

r → 1

w → 1

Activate Windows

Go to Settings to activate Windows

Your Code

Input

SIMATS Online Programming Lab

PYTHON PROGRAMMING



PATAN MOHAMMED RIYAZ KHAN
192572126RC

l → 3

o → 2

r → 1

w → 1

Your Code

```
1 s = input().lower()
2 freq = {}
3
4 for char in s:
5     if char != ' ':
6         freq[char] = freq.get(char, 0) + 1
7
8 for char in sorted(freq):
9     print(char, "->", freq[char])
```

✓ Submitted

Input

hello world

Output

```
d -> 1
e -> 1
h -> 1
l -> 3
o -> 2
r -> 1
```



← Back

Q2 Vowel and Consonant Classifier with Statistics

10 marks

Accept a sentence from the user and display: (a) total vowels and consonants, (b) the most frequent vowel, (c) the least frequent consonant, and (d) the ratio of vowels to consonants as a decimal rounded to 2 places.

Constraints:

- Case-insensitive analysis.
- Ignore spaces and digits.
- Handle sentences with no vowels or no consonants gracefully.

Skills Assessed:

- Character classification with conditionals
- Counting and comparison logic
- Arithmetic and rounding

Sample Input :

Python Programming

Expected Output : Vowels: 5 | Consonants: 12 Most frequent vowel: o (2) Least frequent consonant: y (1) Vowel-to-Consonant Ratio: 0.42



```
1 s = input().strip().lower()
2
3 vowels_set = set("aeiou")
4 vowel_count = 0
5 consonant_count = 0
6
7 vowel_freq = {v: 0 for v in vowels_set}
8 consonant_freq = {}
9
10 for ch in s:
11     if ch.isalpha():
12         if ch in vowels_set:
13             vowel_count += 1
14             vowel_freq[ch] += 1
15         else:
16             consonant_count += 1
17             consonant_freq[ch] = consonant_freq.get(ch, 0) + 1
18
19 most_freq_vowel = None
20 most_freq_vowel_count = 0
21 for v in sorted(vowels_set):
22     if vowel_freq[v] > most_freq_vowel_count:
23         most_freq_vowel = v
24         most_freq_vowel_count = vowel_freq[v]
25
26 least_freq_consonant = None
27 least_freq_consonant_count = 0
28 if consonant_count > 0:
29     least_freq_consonant, least_freq_consonant_count = min(consonant_freq.items(), key=lambda x: x[1])
30
31 ratio = 0.00 if consonant_count == 0 else round(vowel_count / consonant_count, 2)
32
33 print(f"Vowels: {vowel_count} | Consonants: {consonant_count} "
34       f"Most frequent vowel: {most_freq_vowel} ({most_freq_vowel_count}) "
35       f"Least frequent consonant: {least_freq_consonant} ({least_freq_consonant_count}) "
36       f"Vowel-to-Consonant Ratio: {ratio}")
```

Python Programming

Output

```
Vowels: 4 | Consonants: 13 Most
frequent vowel: o (2) Least
frequent consonant: h (1) Vowel-to-
Consonant Ratio: 0.31
```

Activate Windows

Go to Settings to activate Windows



← Back

Q3 String Compression and Run-Length Encoding

10 marks

Implement a Run-Length Encoding (RLE) program that: (a) Compresses a string by replacing consecutive repeated characters with character+count. (b) Decompresses the encoded string back to the original. (c) If the compressed string is not shorter, returns the original.

Constraints:

- Single characters without count still output as single char.
- Handle empty strings.
- Decompressor should handle all valid RLE strings.

Skills Assessed:

- String traversal and comparison
- Count accumulation logic
- String building and integer conversion

Sample Input :

aabbbccdddee

Expected Output

Compressed: a2b3c2d4e2

Decompressed: aabbbccdddee

For input 'abc': Compression not beneficial.

Original:



Your Code

```
1 s = input().strip()
2
3 def compress(x):
4     if x == "":
5         return ""
6     res = []
7     i = 0
8     while i < len(x):
9         j = i
10        while j < len(x) and x[j] == x[i]:
11            j += 1
12        count = j - i
13        if count == 1:
14            res.append(x[i])
15        else:
16            res.append(x[i] + str(count))
17        i = j
18    comp = "".join(res)
19    return comp if len(comp) < len(x) else x
20
21 def decompress(encoded):
22     if encoded == "":
23         return ""
24     res = []
25     i = 0
26     while i < len(encoded):
27         ch = encoded[i]
28         i += 1
29         if i < len(encoded) and encoded[i].isdigit():
30             num = ""
31             while i < len(encoded) and encoded[i].isdigit():
32                 num += encoded[i]
33                 i += 1
34             res.append(ch * int(num))
35         else:
36             res.append(ch)
37     return "".join(res)
38
39 c = compress(s)
40 d = decompress(c)
41
42 print(c)
43 print(d)
```

Input

Output

```
a2b3c2d4e2
aabbccdddee
```

Activate Windows

Go to Settings to activate Windows



← Back

Q4 Advanced Text Statistics Analyser

10 marks

Write a Python program that accepts a paragraph of text and computes: (a) Total words, sentences, and paragraphs. (b) Average word length and average sentence length. (c) The top 5 most frequent words (excluding common stop words: the, is, a, an, in, of, and, to, for). (d) Flesch Reading Ease Score: $206.835 - 1.015 * (\text{words/sentences}) - 84.6 * (\text{syllables/words})$. Approximate syllables as number of vowel groups per word.

Constraints:

- Sentence ends with . ! or ?
- Handle punctuation attached to words.
- Stop words list is hardcoded in the program.

Skills Assessed:

- Complex string parsing and tokenization
- Nested loops for syllable counting
- Floating-point arithmetic and rounding

Sample Input: Python is a powerful programming language

Expected Output :

Words: 45 | Sentences: 4 | Paragraphs: 1

Avg Word Length: 5.2 | Avg Sentence Length: 11.25

Top 5 Words: python(3), programming(2), language(2), data(2), code(1)

Flesch Reading Ease: 58.3 (Standard)



```
1 import re
2 from collections import Counter
3
4 def syllable_count_vowel_groups(word: str) -> int:
5     # Approximate syllables as vowel groups per word
6     # vowel groups: sequences of a,e,i,o,u (case-insensitive)
7     vowels = set("aeiou")
8     w = re.sub(r"^[^a-zA-Z]", "", word).lower()
9     if not w:
10         return 0
11
12     groups = 0
13     prev_is_vowel = False
14     for ch in w:
15         is_vowel = ch in vowels
16         if is_vowel and not prev_is_vowel:
17             groups += 1
18         prev_is_vowel = is_vowel
19
20     return groups if groups > 0 else 1
21
22 def split_sentences(text: str):
23     # Sentence ends with . ! or ?
24     # Treat multiple spaces/newlines safely
25     parts = re.split(r'([.!?])+', text)
26     sentences = []
27     current = ""
28     for i in range(0, len(parts), 2):
29         if i < len(parts):
30             seg = parts[i].strip()
31             punct = parts[i+1] if i+1 < len(parts) else ""
32             if seg or punct:
33                 sent = (seg + punct).strip()
34                 if sent:
35                     sentences.append(sent)
36     return sentences
37
38 def extract_paragraphs(text: str):
39     # Paragraphs separated by one or more blank lines
40     paras = re.split(r'\n\s*\n+', text.strip())
41     return [p.strip() for p in paras if p.strip()]
42
43 def advanced_text_statistics(text: str):
44     stop_words = {
45         "the", "is", "a", "an", "in", "of", "and", "to", "for"
46     }
```

Python is a powerful programming
language

Output

```
Words: 6 | Sentences: 1 |
Paragraphs: 1
Avg Word Length: 6.0 | Avg
Sentence Length: 6.0
Top 5 Words: python(1),
powerful(1), programming(1)
```

Activate Windows

Go to Settings to activate Wind



```
42
43 def advanced_text_statistics(text: str):
44     stop_words = {
45         "the", "is", "a", "an", "in", "of", "and", "to", "for"
46     }
47
48     paragraphs = extract_paragraphs(text)
49     sentences = split_sentences(text)
50
51     # Word tokens: handle punctuation attached to words
52     # Keep alphabetic words only for frequency analysis
53     words_all = re.findall(r"[A-Za-z]+(?:'[A-Za-z-]+)?", text)
54     words_lower = [w.lower() for w in words_all]
55
56     total_words = len(words_all)
57     total_sentences = len(sentences)
58     total_paragraphs = len(paragraphs)
59
60     avg_word_length = (sum(len(w) for w in words_all) / total_words) if
61     avg_sentence_length = (total_words / total_sentences) if total_sen
62
63     # Top 5 frequent words excluding stop words
64     filtered = [w for w in words_lower if w not in stop_words]
65     freq = Counter(filtered)
66     top5 = freq.most_common(5)
67
68     # If fewer than 5, keep what we have; format must still be consist
69     top5_formatted = []
70     for w, c in top5:
71         top5_formatted.append(f"{w} {{c}}")
72     # If none, show empty list representation
73     top5_str = ", ".join(top5_formatted) if top5_formatted else ""
74
75     # Flesch Reading Ease:
76     # 206.835 - 1.015*(words/sentences) - 84.6*(syllables/words)
77     syllables_total = 0
78     for w in words_all:
79         syllables_total += syllable_count_vowel_groups(w)
80
81     if total_words and total_sentences:
82         flesch = 206.835 - 1.015 * (total_words / total_sentences) - 8
83     else:
84         flesch = 0.0
85
86     # Rounding as typically expected in such outputs
87     return {
88         "total_paragraphs": total_paragraphs,
89         "total_sentences": total_sentences,
90         "total_words": total_words,
91         "avg_word_length": avg_word_length,
92         "avg_sentence_length": avg_sentence_length,
93         "top5_frequent_words": top5_str,
94         "flesch_reading_ease": flesch
95     }
```



```
69 top5_formatted = []
70 for w, c in top5:
71     top5_formatted.append(f"{w}({c})")
72 # If none, show empty list representation
73 top5_str = ", ".join(top5_formatted) if top5_formatted else ""
74
75 # Flesch Reading Ease:
76 # 206.835 - 1.015*(words/sentences) - 84.6*(syllables/words)
77 syllables_total = 0
78 for w in words_all:
79     syllables_total += syllable_count_vowel_groups(w)
80
81 if total_words and total_sentences:
82     flesch = 206.835 - 1.015 * (total_words / total_sentences) - 84.6 * (syllables_total / total_words)
83 else:
84     flesch = 0.0
85
86 # Rounding as typically expected in such outputs
87 return {
88     "Words": total_words,
89     "Sentences": total_sentences,
90     "Paragraphs": total_paragraphs,
91     "Avg Word Length": round(avg_word_length, 2),
92     "Avg Sentence Length": round(avg_sentence_length, 2),
93     "Top 5 Words": top5_str,
94     "Flesch Reading Ease": round(flesch, 1),
95 }
96
97 # ----- Main -----
98 if __name__ == "__main__":
99     text = input().strip() # user enters a paragraph
100     stats = advanced_text_statistics(text)
101
102     print(f"Words: {stats['Words']} | Sentences: {stats['Sentences']}")
103     print(f"Avg Word Length: {stats['Avg Word Length']} | Avg Sentence Length: {stats['Avg Sentence Length']}")
104     print(f"Top 5 Words: {stats['Top 5 Words']}")
105     print(f"Flesch Reading Ease: {stats['Flesch Reading Ease']}")
```

Submitted